

PROJECT CONTEXT BRIEF
For continuation in a new AI assistant session

EnigmaGov

Agentic Regulatory Intelligence & Compliance Platform

SuRaksha Cyber Hackathon 2.0 — Theme 2

Canara Bank · Department of Financial Services · Indian Banks' Association

Purpose of this document: This is a complete context brief covering everything decided so far on this hackathon project. It is intended to be read by an AI assistant (or a teammate) so the conversation can continue from exactly where it left off, with no loss of context, decisions, or reasoning.

0. Note to the AI Assistant Reading This

The user (Sajith) is a final-year engineering student building a submission for the SuRaksha Cyber Hackathon 2.0, organised by Canara Bank with the Department of Financial Services and Indian Banks' Association, powered by HackerEarth. He is working on Theme 2: Agentic Regulatory Intelligence & Compliance. Everything below reflects decisions already made in a prior conversation. Treat this as established context — do not re-derive it from scratch, build on it. He values direct, honest, technically detailed answers with clear tradeoffs, and prefers visual/interactive explanations where useful.

1. Hackathon Context

Item	Detail
Event	SuRaksha Cyber Hackathon 2.0
Organisers	Canara Bank, Department of Financial Services, Indian Banks' Association
Platform	Powered by HackerEarth
Theme chosen	Theme 2 — Agentic Regulatory Intelligence & Compliance
Phase 1	Idea submission only — no code required. Deadline May 24, 11:59 PM. Scored out of 50 marks
Phase 2	Working prototype. June 1 to June 30. Scored out of 100 marks. Only shortlisted teams from Phase 1 proceed
Prize pool	Rs. 11 Lakhs total — 1st: Rs. 5L, 2nd: Rs. 3L, 3rd: Rs. 2L, Special prizes: Rs. 50,000

Reference material	An official Inspiration Guide PDF was provided with a sample idea called RegulaMind AI for Theme 2 — used as inspiration only, not to be copied directly
--------------------	--

2. The Problem Being Solved

Banks, NBFCs, insurance companies, and fintech organisations operate under constant regulatory pressure from RBI, SEBI, IRDAI, NPCI, and other bodies. These regulators publish circulars, master directions, amendment notices, FAQs, and supervisory advisories through multiple portals, in different formats, and sometimes in multiple languages.

Currently this is handled manually: compliance teams monitor websites, download PDFs, interpret obligations, assign tasks via email or spreadsheets, and later check whether tasks were completed. This is slow, inconsistent, hard to audit, and does not scale. Missing a single deadline or misinterpreting a circular can cause regulatory penalties and failed inspections.

3. The Solution — EnigmaGov

EnigmaGov is the finalised name for the solution (earlier placeholder names considered during ideation were RegulaMind AI from the inspiration guide and ClarityAI — EnigmaGov is the one being submitted).

EnigmaGov is positioned as a security-first, agentic compliance operating system that automates the entire regulatory lifecycle — from monitoring new circulars to verifying task completion — without manual reading or coordination. It converts regulatory text into structured, trackable, auditable action items called Measurable Action Points (MAPs).

3.1 The three pillars of differentiation

- Security-first — defences against prompt injection, document spoofing, and homoglyph attacks built into every layer, not bolted on afterward. Directly relevant to the cybersecurity theme.
- Accuracy-first — every MAP field is traceable to a quoted source sentence from the original circular, eliminating hallucination risk.
- Confidence-tiered automation — autonomous only when genuinely confident (above 0.90), human-supervised otherwise. Proportional oversight, not blanket automation.

4. Core Concept — Measurable Action Points (MAPs)

A MAP is the fundamental structured output of the system. Every regulatory obligation extracted from a circular becomes one MAP object:

Field	Example	Notes
action	Implement 2FA for transactions above Rs. 50,000	The compliance action required
department	IT	Can be multiple departments for complex obligations
deadline	2025-08-01	Always normalised to ISO 8601 format
deadline_type	explicit / derived / implicit	Tracks whether the date was stated or calculated
priority	High	Critical / High / Medium / Low
obligation_type	REQUIRED	REQUIRED / PROHIBITED / ADVISORY
source_sentence	"All banks must implement two-factor..."	Quoted directly from the circular — hallucination guard
confidence	0.94	0.0 to 1.0 — drives the confidence router
conditions	Applies to scheduled commercial banks	Conditional applicability clauses
circular_id	sha256 hash of source document	Links back to the original document for audit

5. Full System Architecture — All 10 Stages

This is the complete pipeline as finalised, combining the user's own hand-drawn architecture with refinements discussed. Stages run in this order (stages 5 and 6 run in parallel):

Stage 1 — Monitoring Agent

Triggers on a schedule — every 6 hours for standard sources, every 15 minutes for high-priority sources. Watches RBI, SEBI, IRDAI, and NPCI portals using RSS feeds (feedparser), web scraping (Playwright / BeautifulSoup), and notification APIs where available. If nothing new is found, it goes back to sleep until the next trigger. If something new is found, it downloads the document from the trusted domain only and generates a SHA-256 hash immediately.

Stage 2 — Vulnerability / Security Check

Every downloaded document passes through a multi-layer security gate before any AI processing. This is the user's own addition to the architecture and is a key differentiator. Checks run in this order:

- Source domain verification — only rbi.org.in, sebi.gov.in, irdai.gov.in, npcib.org.in are trusted
- TLS certificate validity check
- File type verification using magic bytes, not just file extension (python-magic library)
- SHA-256 hash check against already-processed documents — deduplication and tamper detection

- Content injection scan — checks for embedded scripts, prompt injection phrases (e.g. "ignore previous instructions"), hidden invisible text (white-on-white), Unicode homoglyph attacks, and direction-override characters
- Size and structure sanity check — file size between 10KB and 50MB, minimum extractable text length

On any failure, the document is rejected and logged as a vault complaint — a security incident record with reason, timestamp, and source URL. On pass, it proceeds to structuring.

Stage 3 — Structuring Data

Raw files are converted into clean text depending on format:

- Digital PDF — pdfplumber for layout-aware text extraction
- Scanned PDF — Tesseract OCR with lang="eng+hin", after re-rendering pages to grayscale at 200 DPI (destroys steganographic payloads and reduces attack surface)
- HTML page — BeautifulSoup, with script/style tags removed, HTML comments stripped, and hidden elements (display:none, opacity:0) removed before extracting visible text only

A threshold check then measures extraction quality (ratio of valid alphabetic words to total words). Below 0.5 quality score, the document is flagged for manual review and does not proceed. Above threshold, it is normalised into a standard object structure (id, title, date, source_url, raw_text, language, format, page_count, quality_score) so downstream agents never need to know the original format.

Stage 4 — Orchestrator Agent

Built on LangGraph for stateful, multi-step pipeline management. Uses a fast, cheap LLM (GPT-4o mini or Claude Haiku) purely for routing decisions — it does not do heavy reasoning itself.

Responsibilities: receives the normalised circular object, decides which sub-agents to invoke and in what order, runs the classifier and NLP extractor sub-agents in parallel via asyncio (cuts processing time by roughly 40 percent), and handles retries with exponential backoff (up to 3 retries, then queues for 30-minute retry, then alerts admin) so the pipeline never silently drops a circular.

Stage 5 — Classifier Sub-Agent (runs in parallel with Stage 6)

Model: LEGAL-BERT, fine-tuned on 50 to 100 real RBI circulars, running locally with no API cost.

Identifies: regulation type (cybersecurity, KYC, capital adequacy, data privacy, fraud prevention, etc.), affected department(s), priority level, and language. Chosen over a full LLM because classification is a simple categorisation task that does not need general reasoning — a domain-tuned small model is faster, cheaper, and more accurate on regulatory vocabulary.

Stage 6 — NLP Extraction Sub-Agent (runs in parallel with Stage 5)

Model: spaCy with custom NER pipeline plus LEGAL-BERT embeddings. Extracts custom entity types: OBLIGATION, DEADLINE, CONDITION, ENTITY_TARGET. Uses a sliding window of 3 to 5 sentences (not single-sentence processing) so that obligations spanning multiple sentences are captured fully. Includes a custom implicit deadline resolver for vague regulatory language:

```
IMPLICIT_DEADLINES = {
    "immediately": 7,    # days from circular date
    "forthwith": 3,
    "promptly": 14,
    "at the earliest": 7,
    "without delay": 7,
    "in due course": None, # flag for human review
    "as soon as possible": 14,
}
```

Stage 7 — MAP Generation Sub-Agent

The only sub-agent that calls an external LLM API. Model: GPT-4o mini using OpenAI function calling with a strict JSON schema, with Claude Haiku as automatic fallback if the primary API fails. Combines classifier output and NLP extractor output into structured MAP JSON objects (see Section 4 for the schema). Includes a hallucination guard — every date and obligation must be traceable to the `source_sentence` field; if no exact match is found in the source text, the `deadline_type` is set to "derived" with explicit calculation logic shown, and a Pydantic validator rejects malformed output, triggering a retry with a stricter prompt.

Key pipeline-level fixes identified for this stage: long master circulars (50+ pages) must be chunked into overlapping 2000-token segments with semantic boundaries (split at numbered clause headings like 1.1, 1.2, not arbitrary token counts) to avoid context window overflow silently truncating content. A maximum MAPs-per-circular limit (around 10) prevents the LLM from over-splitting one obligation into many micro-MAPs.

Stage 8 — Confidence Router

Pure rule-based logic, no LLM call, instant decision. Three paths based on the MAP's confidence score:

- Below 0.70 → Human review queue — a compliance officer manually validates before any assignment
- 0.70 to 0.90 → Flag and auto-assign — sent to the department but flagged; department head must confirm before action
- Above 0.90 → Auto-assign directly — sent to department with no human review needed

All three paths converge afterward at the validation sub-agent — every MAP is eventually validated regardless of its routing path.

Stage 9 — Validation Sub-Agent

Model: rule engine plus a lightweight SLM (no heavy LLM needed for most checks). Departments cannot self-report completion. Two types of validation:

- Rule-based — Jira/ServiceNow ticket closure status, policy document hash comparison, system configuration API checks, structured evidence uploads
- Semantic (SLM) — for obligations that cannot be checked with a binary rule, a lightweight model reads the uploaded proof document and checks whether it genuinely addresses the MAP's requirement

Status moves through four states: Pending → In Progress → Marked Done → Verified Done. Only "Verified Done" counts as actual compliance — this two-step model prevents self-reporting fraud. Escalation alerts fire automatically 7 days before deadline if still pending, and escalate to senior management if the deadline passes. Every action is logged to an immutable audit trail.

Stage 10 — Dashboard + Audit Log

Built in React with WebSocket-based real-time updates. JWT authentication with role-based access control (RBAC). Four role-based views:

- Admin — full system visibility, security incident logs (vault complaints), pipeline health metrics, confidence distribution analytics
- Compliance head — all MAPs, department-level compliance health scores, human review queue, escalation panel
- Department head — only their department's MAPs, simple task board, proof upload, deadline countdowns
- Auditor — read-only access to the immutable audit trail covering every action from ingestion to verification, with timestamp and user ID

6. Model and Agent Design Rationale

This was a deliberate architectural decision, not a default choice. The system uses a flat multi-agent design (one orchestrator plus four specialist sub-agents) rather than deep/recursive agents, because the task does not need open-ended recursive research — each sub-agent has one well-defined job.

6.1 SLM vs LLM decision table

Task	Choice	Reason
Classification	SLM — LEGAL-BERT	Simple categorisation task; domain-tuned small model beats general LLM here and is free to run
NER extraction	SLM — spaCy + LEGAL-BERT	Structured, deterministic task; fast and local
MAP generation	LLM — GPT-4o mini	Needs genuine reasoning and structured output generation; only place a full LLM is necessary
Orchestration routing	LLM — Claude Haiku / GPT-4o mini	Only routing logic, kept cheap and fast
Validation rule checks	Rule engine	Binary yes/no checks need no model at all
Semantic validation	Lightweight SLM	Light comparison task, not full reasoning

Rule of thumb established: Use the smallest model that can correctly do the job. Reserve full LLM API calls only for genuine reasoning steps — this keeps the system fast, cheap, and easier to debug.

7. Security Threat Model (Detailed)

Because this is a cybersecurity-themed hackathon, security analysis was done per-document-format, since each format (PDF digital, PDF scanned, HTML, RSS/metadata, embedded tables, bilingual documents) has distinct injection vectors. Summary of the most important threats identified and their defences:

Format	Key threat	Defence
PDF digital	Invisible white-on-white prompt injection text; embedded JavaScript	Strip same-colour text; use pdfplumber/PyMuPDF only, never execute embedded scripts
PDF scanned	Fake RBI letterhead scanned image looks authentic	Source-only ingestion — only accept scanned PDFs fetched directly from official domains
HTML page	Hidden text via display:none / CSS content injection	Strip hidden elements and HTML comments before extracting visible text nodes only
RSS / metadata	Injection in title/description fields; malicious URLs in feed entries	Treat every RSS field as untrusted; whitelist all fetched URLs to official domains
Embedded tables	CSV/formula injection (=CMD(...)) in table cells	Sanitize cells starting with =, +, -, @ before any export; scan each cell individually
Bilingual docs	Hindi-script injection bypassing English filters; Unicode homoglyph attacks (Cyrillic look-alikes)	Maintain multilingual injection pattern lists; run Unicode NFKC normalisation before any pattern matching

A universal pre-LLM safety check was also defined, applied to all extracted text regardless of source format before it reaches any LLM call:

```
def universal_pre_llm_check(text):
    text = unicodedata.normalize("NFKC", text) # homoglyph defence
    text = DIRECTION_CHARS.sub("", text) # RTL confusion defence
    text = re.sub(r'\s{10,}', ' ', text) # hidden spacing defence
    scan_injections(text) # pattern check
    if len(text) < 50: raise ValueError("Extraction failure")
    if len(text) > 500000: raise ValueError("Suspiciously large")
    return text
```

8. Known Pipeline Accuracy Issues (Identified, Not Yet Solved)

An honest accuracy audit was done per pipeline stage. These numbers are realistic estimates, not measured results yet — they exist to set expectations and guide where confidence scoring and human review should be weighted most heavily.

Stage	Estimated accuracy	Biggest failure mode
-------	--------------------	----------------------

Digital PDF extraction	85-92%	Multi-column layouts merge unrelated text
Scanned PDF extraction	60-75%	OCR fails on poor quality scans and mixed Hindi/English
Obligation extraction (NER)	75-82%	Multi-sentence obligations get split into incomplete fragments
Deadline extraction	58-68%	Implicit deadlines ("forthwith", "immediately") are missed entirely — this is the weakest link in the whole pipeline
Department classification (single)	85-90%	Reasonably reliable
Department classification (multi-dept)	50-62%	Circulars affecting multiple departments often get routed to only one
MAP schema compliance	85-92%	Inconsistent date formats from the LLM before normalisation
RAG retrieval relevance	70-78%	Chunk boundaries break context; outdated circulars sometimes outrank current ones

Key takeaway from this audit: 100% accuracy is not achievable or expected. The system's real strength is knowing when it is uncertain (via confidence scoring) and routing those cases to humans — that is the actual engineering achievement, more impressive to judges than a false claim of perfect accuracy.

9. Honest Audit of the Current Approach

9.1 What is genuinely strong

- Multi-agent design — focused, testable, independently replaceable sub-agents
- SLM + LLM split — cost-efficient, most teams would burn tokens using a full LLM for everything
- Confidence routing — mirrors how real compliance teams handle uncertainty
- Security depth — rare in hackathon submissions, directly ties to the cybersecurity theme
- Source-cited MAPs — genuine innovation, makes the system auditable

9.2 Current weaknesses identified

- No feedback loop — human corrections to wrong MAPs are not currently captured as training signal; the same mistake would repeat
- Amendment handling is shallow — amendment language is detected but the actual diff between old and new obligations is not yet resolved
- Validation is mostly manual — the "agentic" validation claim is weak until real automated checks (Jira API, hash comparisons) are wired in
- No observability — a silently failing pipeline stage would currently go unnoticed, which is dangerous in a compliance context

- Single point of failure on the LLM API — no fallback chain was implemented yet at the time of this audit (fallback to Claude Haiku was proposed but needs to be built)

9.3 Specific improvements proposed

- Run classifier and NLP extractor in parallel (asyncio.gather) instead of sequentially — roughly 40% faster
- Add a sentence relevance re-ranker before MAP generation so irrelevant preamble/background text is filtered out before reaching the LLM
- Add MAP deduplication across circulars — semantic similarity check (>85% similar) merges near-duplicate MAPs from multiple circulars on the same topic instead of creating redundant ones
- Use semantic chunking at numbered clause boundaries (1.1, 1.2, 2.0) instead of arbitrary token-count chunking

9.4 New feature ideas proposed (not yet built)

- Regulatory impact scoring — weight MAPs by (customers affected × penalty amount × urgency), not just deadline proximity
- Cross-circular conflict detection — flag when two circulars from different dates give contradictory instructions on the same rule
- Deadline risk predictor — use historical completion-time data per department to flag MAPs likely to miss their deadline early, not on the last day
- Plain-language MAP summaries — a non-technical "what this means for you" summary alongside the formal MAP, since RBI legal language is dense for non-compliance staff
- Department compliance health score — a rolling 0-100 score per department (on-time completions ÷ total MAPs × recency weight) for leadership visibility

10. Complete Technology Stack

Layer	Technology	Purpose
Monitoring	APScheduler + feedparser + Playwright	Scheduled checks, RSS parsing, JS-rendered page scraping
Security checks	python-magic + hashlib + custom scanner	Magic byte verification, SHA-256 dedup, injection detection
PDF extraction	pdfplumber + Tesseract OCR	Digital and scanned PDF text extraction, bilingual OCR
HTML extraction	BeautifulSoup	Safe text extraction stripping scripts/hidden elements
Classification	LEGAL-BERT (local)	Domain-tuned, no API cost, fast categorisation
NLP extraction	spaCy + custom NER	Custom entities: OBLIGATION, DEADLINE, CONDITION

MAP generation	GPT-4o mini (function calling)	Structured JSON MAP output, only LLM API call in pipeline
Fallback LLM	Claude Haiku	Used if primary LLM API is unavailable
Agent orchestration	LangGraph	Stateful multi-step pipeline, conditional routing, retries
Vector DB	ChromaDB	RAG retrieval, semantic dedup, conflict detection
Structured DB	PostgreSQL	MAPs, audit logs, users, status tracking
Document DB	MongoDB	Raw circular storage
Backend API	Python FastAPI	REST endpoints, async, built-in Pydantic validation
Frontend	React + WebSocket	Real-time role-based dashboard
Auth	JWT + bcrypt + RBAC	Login, session timeout, role separation
Workflow engine (optional)	Temporal	Long-running task lifecycle tracking
Deployment	Docker + AWS / Azure	Containerised microservices

11. Current Status and Next Steps

11.1 What has been completed so far

- Problem statement and solution concept finalised — EnigmaGov
- Full 10-stage system architecture designed, combining the user's own hand-drawn diagram with refinements
- Model selection completed for every stage with documented reasoning (SLM vs LLM decision table)
- Security threat model completed in detail, per document format
- Pipeline accuracy issues identified and documented honestly per stage
- Improvement opportunities and new feature ideas documented
- Two full draft versions of the Phase 1 idea-submission write-up were produced and reviewed

11.2 What is needed next

- Finalise and submit the Phase 1 idea write-up on HackerEarth before the deadline (May 24, 11:59 PM)
- If shortlisted for Phase 2, begin actual implementation — starting priority order: PDF/text extraction pipeline, then LLM MAP generation with structured output, then department classifier, then FastAPI + PostgreSQL backend, then React dashboard, then bonus features (knowledge graph, conflict detection)
- Build the implicit deadline resolver and the amendment/versioning diff engine — flagged as the two most critical unresolved technical pieces

- Decide on team member roles and division of work for Phase 2 if the team is more than one person

Instruction for continuation: When the user returns to this project in a new session, they will likely want help with: refining the Phase 1 submission text further, starting actual code for the prototype (PDF extraction, LangGraph pipeline, FastAPI backend, React dashboard), or going deeper into any single stage above. Use this document as the source of truth for what has already been decided rather than asking the user to re-explain the project from scratch.